

IntelliOps4

Intelligent operations for SAP S/4HANA. A chat window over live SAP data that answers in plain language, watches for what you did not ask about, and refuses to change anything without you.

Product and capability reference · 6 September 2026

1 · The problem

A supervisor needs one number. How much of a material is on hand. Whether a purchase order has been received. Which inventory counts are still open.

That number already exists. It is in S/4HANA and always has been. The difficulty was never missing data — it is that reaching it costs a transaction code, a screen someone was trained on, and often a colleague who knows which of fourteen fields matters.

So the number gets asked for instead of looked up. A message. A phone call. A spreadsheet that was accurate on Tuesday. And the decision waits.

PAIN	WHAT INTELLIOPS4 DOES ABOUT IT
Data is behind a training barrier	Ask in your own words. No transaction code, no screen to learn.
Nobody trusts an answer they cannot check	Every answer carries its sources: which system replied, how many rows, and the exact query sent.
An AI that guesses is worse than no AI	Every answer is marked <i>grounded</i> or not. Grounded means a real SAP query returned rows.
Nobody will let software change stock	A write is described and stopped. A person approves it as a separate step.
You only find out after it has gone wrong	It watches on a schedule and tells you — the morning digest, thresholds, anomalies.
Nobody can predict the bill	Per-user quota checked before anything is spent; repeated questions served from cache.
Procurement asks where the data goes	Runs on SAP BTP with your identity, and can route to your own SAP AI Core tenant.

2 · What it does

Answering

- **Ask in plain language.** Five business objects live today — material stock, goods movements, physical inventory, deliveries, purchase orders. Extending to another area of S/4 is a configuration row, not a project.
- **Table or chart, one click.** Same figures, two renderings, no second source of truth.
- **Sources on every answer.** Which system replied, how many rows came back, and the query that was sent — so an empty answer can be told apart from a wrong plant.
- **Grounded / not grounded.** Correct for a greeting; a red flag on a stock question.

- **Conversation memory.** Follow-up questions work without repeating yourself.
- **Periods, not just today.** **BETA** Nine presets — last 7 days, this week, last month and so on.
- **Why, not only what.** **BETA** A causal question chains several queries — what moved, then what was expected — and says plainly when the data does not explain it.
- **Comparison.** **BETA** This week against last, two queries and the difference. Never a trend inferred from one reading.
- **What-if.** **BETA** “If I move 500 out, what runs short?” — always labelled a projection, never presented as a figure from SAP.
- **Questions that take minutes.** **BETA** Queue a slow question, collect the answer later, instead of hitting the 75-second request ceiling.

Acting — always behind a human

- **A write is proposed, never executed.** It is described, costed and audited, and stops.
- **Incomplete instructions are questioned.** A missing quantity is asked for, not defaulted to 1.
- **Anomalous moves are held.** A quantity far outside recent norms goes to a human whatever the policy says.
- **Write ceilings and second approvers.** Configurable per user, warehouse or organisation.
- **Several writes, one approval.** **BETA** Each keeps its own checks; the flagged count is shown on the batch.

Watching — the part that acts on its own **BETA**

- **Morning digest.** Open orders, counts, deliveries and yesterday’s activity, at an hour you choose.
- **Watchers.** “Tell me if stock of P123 drops below 500.” Alerts on *entering* breach, not while it lasts — so it never becomes noise you mute.
- **Anomaly detection.** Compares each object against its own 14-day median, in both directions. Silent until it has five observations, so it does not cry wolf.
- **Delivery.** One webhook that Teams, Slack and any custom endpoint all accept.

Governing

- **Quota per user, role or default** — daily, weekly and monthly, checked before spending.
- **Answer cache** with per-object freshness and a midnight clamp for date-bound answers.
- **Full audit.** One row per request on every path, including refusals.
- **14 admin screens** for business objects, endpoints, cache, quotas, model routes, users, approvals, audit — plus four new ones for background work.
- **Feedback.** **BETA** Thumbs up or down, keyed to the audit row so a rating joins to the model, tools and grounding that produced the answer.
- **Saved questions.** **BETA** A vetted library per role.
- **Preferences.** **BETA** Remembers your plant — from an allowlisted set of structured values only, never free text.
- **Scheduled exports.** **BETA** Usage, quality and failures as CSV.

3 · One day, every feature

A single working day at a manufacturer running S/4HANA. Every capability above appears once, in the order someone would actually meet it.

06:00 — before anyone arrives

The **scheduler** wakes on one instance only — the database lease decides which — and runs the **morning digest**. It reads through the same tools the chat uses, so it cannot disagree with what the chat would say. The **webhook** posts it into the team's Teams channel:

"3 purchase orders open · 2 physical inventory documents still open · 41 goods movements recorded · Yesterday: 27 questions asked, 24 answered from data, 6 served from cache, none failed."

One section could not be read this morning, and says so rather than showing zero — because an empty result and a broken connection lead to opposite actions.

07:15 — Priya opens the chat

The **saved question library** offers her role's usual starting points. She picks *"How much stock do we have?"* Her **preference** already holds plant 1710, so she does not restate it. The answer arrives as a **table**; she switches to a **chart** with one click. It is marked **grounded**, and expanding **Sources** shows SAP Graph, 214 rows, and the exact query.

07:20 — the same question again

A colleague asks it too. The **cache** serves it in 3 milliseconds instead of 900 — no model call, no SAP call. The entry expires on that object's own policy, and a date-bound answer cannot outlive the day it describes.

09:40 — a watcher fires

Stock of P123 crosses below 500. The **watcher** alerts *once*, on entering breach. It stays breached for the rest of the morning and says nothing more — which is why the alert is still trusted a month later.

10:05 — "why?"

Priya asks *"Why did stock drop in 1710 over the last 7 days?"* The router reads it as a lookup and starts on the light model. Two rounds in, the question is still going — so **adaptive escalation** promotes the remaining rounds to the heavier model, because difficulty is discovered, not predicted. The **root-cause chain** queries goods movements over the **date range**, then purchase orders, and connects them. The audit row records the escalation, so *"why did that one cost more?"* has an answer.

10:30 — a projection

She asks *"What if I move 500 out of 1710 — would anything run short?"* The **simulation** reads real stock and does arithmetic on it. The answer leads with *"Projection — arithmetic on the current reading, not a figure from SAP"*, shows the table, and says what it does not model: reservations, safety stock, open

orders, lead times. It is **not marked grounded**, and it never reaches the approval queue — a question nobody can approve by accident.

11:00 — a change

She asks to rebalance five materials. All five **writes are collected**, each judged on its own for ceiling and anomaly, and presented as one **batch** with the flagged count on it. One is unusually large and is marked. She approves. Each is executed through the same single-action path — consumed exactly once — and the answer says plainly whether SAP actually changed.

11:40 — a slow question

She asks for a month of movements across every plant. That is legitimately slower than a browser will wait, so it is **queued**. A run id comes back immediately, a background worker claims it conditionally, runs the *same* agent with a ten-minute budget, and reconciles the quota. If the container is recycled mid-run, the run is expired and the quota returned rather than leaving a spinner that never resolves.

13:00 — anomaly

The **anomaly sweep** compares each object against its own 14-day median. Deliveries are a third of normal. It alerts in *both* directions — a collapse matters as much as a spike — using a median so that one freak day does not teach it that freak days are normal.

14:00 — quality

Priya marks one answer down. The **rating** is keyed to the audit row, so it joins to the provider, model, tools and grounding of that answer. “83% positive” tells nobody anything; “every negative rating this week was ungrounded” is a finding.

16:00 — the quota holds

A heavy user reaches their daily limit. They are refused before anything is spent, with the window that was breached and when it resets. Two OpenRouter keys mean a free tier exhausted at noon fails over rather than collapsing to the offline provider mid-demo.

Friday 07:00 — the export

The **weekly export** builds three CSVs — usage, quality, failures — and posts them on. It is off until someone asks for it, because a weekly file nobody requested is the definition of noise.

4 · How it refuses to be wrong

Most of the engineering is not in answering. It is in the difference between an answer and a confident guess.

Grounded means a real system did something

A read from SAP grounds an answer. A projection does not. A write recorded locally but not posted to SAP does not. All three would otherwise sit behind the same badge.

Unreadable is never treated as fine

A digest section that could not be read says so rather than showing zero. A watcher that cannot be read stays breached rather than clearing a live alert. A failed observation is not stored as a zero, because that would distort what counts as normal from then on.

Nothing changes a real system unattended

Writes stop for approval. Batches do not weaken the per-action checks. Background jobs never execute a write at all — nobody is watching them.

Remembering is not the same as remembering what you said

Preferences are an allowlisted set of structured values with a validator each. Nothing free-text can be stored, so nothing a user writes can reach the system prompt — where grounding and write-refusal live.

5 · What it runs on

LAYER	CHOICE
Platform	SAP BTP, Cloud Foundry
Application	SAP Cloud Application Programming model (CAP), Node.js
Screens	SAP Fiori elements, Horizon theme
Identity	XSUAA — the sign-on and role collections your administrators already assign
SAP data	SAP Graph on your own tenant; Business Accelerator Hub for the rest
System of record	PostgreSQL — users, quotas, config, audit, chat history
Cache	Redis. Optional: without it, an in-process cache
Models	Free OpenRouter models with two-key failover; SAP AI Core for a client who needs it
Portability	Postgres and Redis run on AWS, GCP or Azure — probes verify a client instance before deploying

6 · Status, honestly

Everything in section 2 is built, tested and deployed. Features marked **BETA** were added in the agentic build of September 2026 and are new surface: they work and are covered by tests, but have not yet run for months in front of users.

Two roadmap items were deliberately not built, each with a written trigger for revisiting:

- **Domain agents** — all business objects currently sit in one domain, so a router would choose between one option. Build it when objects span two or more domains, or the tool count passes roughly 15–20.
- **Additional write-backs** — no write currently reaches SAP, because the Accelerator Hub sandbox this tenant reads from is read-only. Adding more would add tools that report success and change nothing. Unblocks with a writable tenant.

Multi-tenancy is not built. The product is single-tenant: one deployment per client. That is a deliberate open decision, not an oversight.